To deploy a Laravel application on an Apache server running Debian 12, you need to set up a **LAMP stack (Linux, Apache, MySQL, PHP)**, deploy your application code, and configure an Apache virtual host.

Prerequisites

- A server running Debian 12.
- SSH access to the server.
- A domain name pointed to the server's IP address (optional but recommended for a production setup).
- Basic knowledge of Linux commands.

Step-by-Step Deployment Guide

1. Update System Packages and Install Apache

First, connect to your server via SSH and update the system.

bash

sudo apt update && sudo apt upgrade -y

Install Apache and ensure it is running:

bash

sudo apt install apache2 apache2-utils -y

sudo systemctl enable apache2

sudo systemctl status apache2

You can verify Apache is working by navigating to your server's IP address in a web browser.

2. Install PHP and Required Extensions

Laravel requires PHP 8.2 or higher (Laravel 12 specifically requires PHP 8.2) and several PHP extensions.

bash

sudo apt install php libapache2-mod-php php-cli php-fpm php-json php-pdo php-mysql php-zip php-gd php-mbstring php-curl php-xml php-pear php-bcmath -y

Enable the Apache PHP module and restart Apache:

bash

sudo a2enmod php8.2

sudo systemctl restart apache2

3. Install a Database Server (MariaDB/MySQL)

Install MariaDB or MySQL for your database needs:

bash

sudo apt install mariadb-server mariadb-client -y

sudo mysql_secure_installation

During the mysql_secure_installation process, you will be prompted to set a root password, remove anonymous users, and disable remote root login.

4. Install Composer

Composer is used to manage Laravel's dependencies.

bash

sudo apt install composer -y

5. Deploy Your Laravel Application

Place your Laravel project files in the /var/www/ directory. The recommended method is to use Git to clone your repository.

bash

cd /var/www/

sudo git clone <your_repository_url> <your_app_name>

Navigate into your project directory and install the PHP dependencies:

bash

cd /var/www/<your_app_name>

composer install --optimize-autoloader --no-dev

6. Configure Environment and Permissions

Copy the example environment file and generate an application key:

bash

cp .env.example .env

php artisan key:generate

Edit the .env file to configure your database credentials and other settings.

bash

nano .env

 # Update DB_DATABASE, DB_USERNAME, DB_PASSWORD fields

Ensure the correct file permissions are set for the web server user (www-data on Debian/Ubuntu):

bash

sudo chown -R www-data:www-data /var/www/<your_app_name>

sudo chmod -R g+w /var/www/<your_app_name>/storage

Finally, run the database migrations:

bash

```
php artisan migrate
```

7. Configure Apache Virtual Host

Create a new virtual host configuration file for your Laravel application. Disable the default site first:

bash

```
sudo a2dissite 000-default.conf
sudo nano /etc/apache2/sites-available/your_app_name.conf
```

Add the following configuration, replacing your_app_name and yourdomain.com with your details:

apache

```
<VirtualHost *:80>
    ServerAdmin admin@yourdomain.com
    ServerName yourdomain.com
    ServerAlias www.yourdomain.com
    DocumentRoot /var/www/your_app_name/public

    <Directory /var/www/your_app_name/public>
        Options Indexes FollowSymLinks MultiViews
        AllowOverride All
        Order allow,deny
        allow from all
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Enable the new site, enable the rewrite module (if not already enabled), and restart Apache:

bash

```
sudo a2ensite your_app_name.conf
sudo a2enmod rewrite
sudo systemctl restart apache2
```